

Graphes

1. Idée générale et analogie

Un graphe est une façon de modéliser des **relations entre des entités** quelconques. Dans ce contexte, les entités sont appelées **sommets** et les relations **arêtes** (ou **arcs**, selon le type de graphe).

L'analogie la plus parlante est celle d'un **réseau routier** : les *villes* sont les *sommets*, les *routes* sont les *arêtes*. Aller de Paris à Lyon ne signifie pas forcément qu'on peut faire *exactement* le **même chemin** à l'envers (il peut y avoir des sens interdit, des routes à sens unique...) : c'est là qu'intervient la notion d'**orientation**.

⚠ Limites de l'analogie

Un réseau routier est presque toujours **connexe** (on peut aller de n'importe quelle ville à n'importe quelle autre). Un graphe quelconque ne l'est pas forcément : certains sommets peuvent être *isolés* ou *inaccessibles* depuis un sommet donné. Mieux vaut, donc, *ne pas* généraliser l'intuition « tout est relié ».

2. Définition et vocabulaire

2.1. Définition

Formellement, un **graphe** est un couple (S, A) où :

- S est un ensemble fini de **sommets** (parfois appelés *nœuds*, comme pour les *arbres*) ;
- A est un ensemble de **liaisons** entre certains sommets.

Adaptation du vocabulaire à l'orientation du graphe :

	Graphe non orienté	Graphe orienté
Liaison	Arête	Arc
Notation	$\{u, v\}$ (paire non ordonnée)	(u, v) (couple ordonné)
Symétrie	Si $u \rightarrow v$ alors $v \rightarrow u$	On peut avoir $u \rightarrow v$ mais pas $v \rightarrow u$
Exemple	Réseau d'amis	Réseau routier (sens uniques possibles)

!! Un graphe peut être **pondéré** : chaque arête ou arc porte alors un **poids** (qui représente une distance, un coût, une durée...).

⚠ Ces poids peuvent être *négatifs*.

✅ Truc mnémotechnique

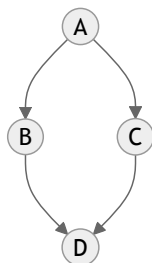
Un **arc**, ça ne va pas sans **flèches** ! Cela permet de se souvenir que dans un graphe *orienté*, les liens (alors représentés par des *flèches*) sont appelés *arcs*.

⚠ Attention

Un sommet peut n'avoir aucune liaison : on parle alors de sommet **isolé**. C'est *parfaitement valide* : la définition n'impose **pas** que tous les sommets soient reliés.

2.2. Exemple

Voici un graphe orienté simple. Il sera réutilisé dans la suite de ce document.



2.3. Vocabulaire associé

- **Voisin** de u : tout sommet v tel qu'il existe une liaison de u vers v . Dans un graphe orienté, on parle plutôt de *successeur* (à ne pas confondre avec *prédécesseur* : un sommet *depuis lequel* on peut atteindre u).
- **Degré d'un sommet** : nombre de liaisons directement connectées à ce sommet. Dans un graphe orienté, on distingue le degré entrant et le degré sortant.
- **Chemin** : suite de sommets reliés deux à deux par des arêtes ou arcs (dans un graphe orienté, il *faut* respecter le sens des arcs suivis).
- **Cycle** : chemin dont le sommet de départ et le sommet d'arrivée sont identiques.
- **Ordre d'un graphe** : nombre de sommets du graphe (noté $|S|$ ou n).
- **Taille d'un graphe** : nombre d'arêtes ou d'arcs du graphe (noté $|A|$ ou m).
- **Graphe connexe** : graphe non orienté dans lequel *tout sommet* est accessible depuis *tout autre sommet*.

Remarque : degrés entrant et sortant ▼

On dit qu'une liaison est *incidente* à un sommet lorsqu'elle le relie directement à un autre sommet — qu'elle y arrive ou en parte. **Dans un graphe orienté**, on affine cette notion selon la *direction* de l'arc :

- **degré entrant** (*in-degree*) : nombre d'arcs *incidents entrants*, c'est-à-dire qui arrivent à ce sommet (combien de sommets « pointent vers lui ») ;
- **degré sortant** (*out-degree*) : nombre d'arcs *incidents sortants*, c'est-à-dire qui partent de ce sommet (combien de sommets il « pointe »).

Sur l'exemple ci-dessus : le sommet **D** a un degré entrant de 2 (**B**→**D** et **C**→**D**) et un degré sortant de 0 (aucun arc ne part de **D**).

Remarque : lien avec les arbres ▼

Pour la culture

Un graphe **non orienté, connexe et sans cycle** est aussi appelé un *arbre libre*. Choisir un sommet comme **racine** en fait un *arbre enraciné* — la forme habituelle en NSI. Un même arbre libre peut donner autant d'arbres enracinés *différents* qu'il a de sommets.

3. Représentation en Python

Deux représentations sont au programme.

3.1. Liste d'adjacence

En pratique, on utilisera souvent un **dictionnaire** Python dont les clefs sont les sommets et les valeurs sont les *listes* de voisins. Pour le graphe de l'exemple ci-dessus, cela donne :

```
# Graphe orienté : A→B, A→C, B→D, C→D
graphe = {
    'A': ['B', 'C'],
    'B': ['D'],
    'C': ['D'],
    'D': []
}
```

- **Accès aux voisins de `u`** : `graphe[u]` ou `graphe.get(u)`.
- **Avantage** : efficace pour les graphes *creux* (peu d'arêtes entre les sommets).

Remarque : méthode `get` des dictionnaires Python ? ▼

- Si `u` n'est pas un sommet du graphe (donc une clef du dictionnaire le représentant), l'instruction `graphe[u]` lève l'exception (provoque l'erreur) `KeyError`.
- Pour du code plus défensif, `graphe.get(u)` renvoie `None` si `u` est absent du graphe. Ce `None` peut alors être détecté et traité explicitement.
- Variante, à n'employer qu'en toute conscience : `graphe.get(u, [])` renvoie `[]` si `u` est absent du dictionnaire. Tout se passe comme si ce sommet existait dans le graphe, mais *sans voisin*. C'est utile lorsqu'on décide de représenter un graphe sans déclarer explicitement les sommets sans voisins (sortants). Mais cela masque *silencieusement* tout sommet **vraiment absent** (suite à une erreur de déclaration, par exemple).

3.2. Matrice d'adjacence

C'est un tableau à double entrée. Il faut imaginer qu'on a numéroté les sommets : par exemple **A↔0**, **B↔1**, **C↔2**, **D↔3** etc. On place alors *mentalement* ces sommets face à chaque ligne (dans l'ordre de la numérotation, du haut vers le bas) et chaque colonne (toujours dans l'ordre, de la gauche vers la droite).

Ensuite, pour chaque sommet numéroté *i*, on met **1** dans la case du tableau à la ligne *i* et la colonne *j* s'il existe une liaison du sommet *i* vers le sommet *j*, **0** sinon.

	A (j=0)	B (j=1)	C (j=2)	D (j=3)
A (i=0)	0	1	1	0
B (i=1)	0	0	0	1
C (i=2)	0	0	0	1
D (i=3)	0	0	0	0

En Python, ce tableau — appelons-le **matrice** — prendra la forme d'une **liste de liste** : `matrice[i]` désigne la *i*-ème ligne (comptée à partir de **0**), `matrice[i][j]` désigne la *j*-ème case de cette ligne (là aussi comptée à partir de **0**). Ainsi, `matrice[i][j]` vaut **1** s'il existe une liaison du sommet *i* vers le sommet *j*, **0** sinon.

Pour le graphe de l'exemple ci-dessus, cela donne :

```
matrice = [
#   A  B  C  D      # Destinations
[0, 1, 1, 0],      # Origine A : A→B et A→C      (ligne 0 : i=0)
[0, 0, 0, 1],      # Origine B : B→D          (ligne 1 : i=1)
[0, 0, 0, 1],      # Origine C : C→D          (ligne 2 : i=2)
[0, 0, 0, 0],      # Origine D : aucun arc sortant (ligne 3 : i=3)
]
```

Explicitation :

- `matrice[0][1]` vaut `1` : il existe un arc A→B.
- `matrice[3][0]` vaut `0` : il n'existe pas d'arc D→A.

Remarque : deux précisions ▼

- **Pour un graphe non orienté**, la matrice est **symétrique** : `matrice[i][j] == matrice[j][i]`. Concrètement, cela signifie que les coefficients sont symétriques par rapport à la *première diagonale* de la matrice (celle qui part d'en haut à gauche et s'achève en bas à droite).
- **Occupation potentiellement importante de l'espace mémoire** : il y a S^2 valeurs à mémoriser quand S désigne le nombre de sommets — c'est coûteux en espace pour les grands graphes creux (beaucoup de sommets, peu de liaisons entre eux).

3.3. Points de vigilance ⚠

- `matrice[i][j]` : `i` est l'indice de **ligne** (sommets source), `j` est l'indice de **colonne** (sommets destination).
⚠ **Ne pas les inverser** 🧠 !
- La liste d'adjacence utilise les **noms** des sommets comme clefs ; la matrice utilise des **indices entiers**. Les deux représentations ne se manipulent pas de la même façon.

4. Parcours de graphes

4.1. Principe commun

Un parcours visite tous les sommets **accessibles** depuis un sommet de départ. *N'importe quel sommet* d'un graphe peut être le sommet de départ, lors d'un parcours. C'est une différence avec les arbres (le point d'entrée *naturel* d'un arbre est sa racine).

⚠ 🧠 Piège majeur : les cycles

Contrairement aux arbres, un graphe peut contenir des **cycles**. Sans mécanisme pour se souvenir des sommets déjà visités, un parcours peut boucler indéfiniment. Il est **indispensable** de maintenir un « ensemble » des sommets déjà vus (en NSI on utilise souvent un *dictionnaire Python* à cette fin).

Remarque : structure abstraite de données « ensemble » ▼

Python dispose d'une structure de données nommée `set` : ajouter un élément à un `set` ne modifie l'ensemble *que* si cet élément est *nouveau*. Mais `set` n'est pas explicitement au programme...

Il existe deux stratégies de parcours :

	DFS (<i>Depth First Search</i>)	BFS (<i>Breadth First Search</i>)
Traduction en Français	Parcours en profondeur <i>d'abord</i>	Parcours en largeur <i>d'abord</i>
Structure auxiliaire	Pile (explicite ou pile d'appels pour un parcours récursif)	File
Implémentation naturelle	Récursive	Itérative
Propriété	Explore loin avant de revenir	Explore par « couches » successives
Chemin le plus court	Non garanti	Garanti (en nombre d'arêtes)

 **Remarque : pourquoi « d'abord » ? ▼**

En général, on abrège : on dit juste « parcours en profondeur » ou « parcours en largeur ». Mais ces notions peuvent paraître délicates à bien cerner : parcourir un graphe implique en effet de se déplacer « dans les deux dimensions ».

- Un parcours en profondeur éloigne initialement très vite du sommet de départ, mais quand cela n'est plus possible le parcours se poursuit nécessairement par un « déplacement en largeur » avant de reprendre en profondeur.
- Un parcours en largeur s'attache à explorer méthodiquement chaque voisin, puis les voisins de ces voisins, etc. Il s'accompagne donc aussi d'une forme de déplacement « en profondeur ».

Avec DFS, « en profondeur d’abord » signifie qu’après la visite d’un sommet, on aborde un sommet plus profond avant un sommet de même profondeur.

Avec BFS, « en largeur d’abord » signifie qu’après la visite d’un sommet, on aborde un sommet de même profondeur avant un sommet plus profond.

4.2. DFS : parcours en profondeur

Le DFS est **naturellement récursif** : on explore un voisin à fond avant de passer au suivant.

```
def dfs(sommet, graphe, vu):  
    """Parcours en profondeur depuis `sommet`. """  
    if vu[sommet]:          # sommet déjà visité : on arrête cette branche  
        return             # return explicite (équivalent à return None)  
    vu[sommet] = True  
    print(sommet)  
    for voisin in graphe[sommet]:  
        dfs(voisin, graphe, vu)
```

Utilisation :

```
# Il FAUT définir `vu` à l'extérieur de `dfs` ! Voir la remarque ci-dessous...  
vu = {sommet: False for sommet in graphe}  # graphe : celui donné en exemple plus haut  
dfs('A', graphe, vu)
```

Schéma récursif :

- **cas de base** : sommet déjà visité → `return` ;
- **appel récursif** : on appelle `dfs` sur chaque voisin non encore visité. Ainsi, le 1^{er} voisin de **A** est **B** : le parcours récursif se poursuit avec **B**, dont 1^{er} voisin est **D**, etc.
- **effet de bord** : `vu` est partagé entre tous les appels (c'est voulu).

 **Remarque : DFS itératif avec pile ▼**

On peut aussi réaliser un DFS **itératif** en gérant explicitement une pile (liste Python). L'ordre de visite peut différer légèrement du DFS récursif selon l'ordre d'empilement des voisins. Le code ci-dessous fait *justement* en sorte que cet ordre ne diffère pas (mais cela a un coût).

```
def dfs_iter(depart, graphe):
    vu = {sommet: False for sommet in graphe}
    pile = [depart]
    while pile:
        sommet = pile.pop()          # Dépile le dernier sommet entré
        if not vu[sommet]:
            vu[sommet] = True
            print(sommet)
            # reversed() pour conserver le même ordre de visite que le DFS récursif
            for voisin in reversed(graphe[sommet]):
                if not vu[voisin]:
                    pile.append(voisin)
```

Ici, `vu` peut être défini à l'intérieur de la fonction : sans récursivité, il n'y a aucun risque de réinitialisation.

⚠ Piège

Le dictionnaire `vu` **DOIT** être défini en dehors de `dfs()` ! S'il était créé à l'intérieur, chaque appel récursif le réinitialiserait. Pour mieux comprendre, imaginons qu'on l'ait défini à l'intérieur :

```
def dfs_faux(sommet, graphe):
    vu = {sommet: False for sommet in graphe} # ERREUR
    if vu[sommet]:
        return
    vu[sommet] = True
    print(sommet)
    for voisin in graphe[sommet]:
        dfs_faux(voisin, graphe)
```

À chaque appel récursif, un nouveau dictionnaire `vu` est créé, entièrement (ré)initialisé à `False`. Le marquage du sommet visité est immédiatement perdu : sur un graphe avec cycle, c'est une boucle infinie assurée ! Même sans cycle, les sommets seraient revisités et affichés plusieurs fois...

Attention aux arguments mutables dans les appels récursifs et, plus généralement, être très prudent lorsqu'une structure de données (liste, dictionnaire, ensemble...) doit être partagée et mise à jour.

💬 Remarque : pour aller plus loin ▼

Une solution élégante existe :

```
def dfs(sommet, graphe, vu=None):
    if vu is None: # premier appel : on crée `vu` une seule fois
        vu = {s: False for s in graphe}
    if vu[sommet]:
        return
    vu[sommet] = True
    print(sommet)
    for voisin in graphe[sommet]:
        dfs(voisin, graphe, vu) # `vu` est transmis à chaque appel récursif
```

Le `if vu is None` ne s'exécute qu'au premier appel (celui de l'utilisateur). Les appels récursifs reçoivent le même dictionnaire `vu`, déjà initialisé et progressivement rempli.

4.3. BFS : parcours en largeur et recherche de chemin

Le BFS explore les sommets **couche par couche** (*tous* les voisins directs, puis leurs voisins, etc.). Il progresse en « cercles » concentriques, « centrés » sur le sommet de départ.

Le parcours BFS garantit de trouver le **chemin le plus court en nombre d'arêtes** entre le départ et la cible. C'est donc un algorithme de choix pour rechercher s'il existe un chemin entre deux sommets.

Il se réalise de façon **itérative** avec une **file**.

```
def bfs_chemin(depart, cible, graphe):
    """Renvoie le plus court chemin (en arêtes) de `depart` à `cible`
    ou [] si `cible` est inaccessible depuis `depart`."""
    # Dictionnaire pour garder la trace des sommets déjà rencontrés
    vu = {sommet: False for sommet in graphe}
    # Dictionnaire qui associe à chaque sommet-clef, un sommet-parent
    parent = {sommet: None for sommet in graphe}
    fil = [depart] # Une liste utilisée comme file
    vu[depart] = True

    while fil:
        # Tant que `fil` n'est pas vide
        sommet = fil.pop(0) # on défile le 1er entré ⚠ pop(0) lent
        # Ici, on a atteint la cible !
        if sommet == cible:
            break # Donc on interrompt la boucle
        for voisin in graphe[sommet]: # Sinon on explore les voisins
            if not vu[voisin]:
                vu[voisin] = True
                parent[voisin] = sommet # On mémorise par où on est arrivé
                fil.append(voisin) # On enfile le voisin découvert

    if not vu[cible]:
        # `cible` jamais atteinte
        return []

    # Reconstruction du chemin en remontant les parents de `cible`
    chemin = []
    courant = cible
    # Cette boucle termine car parent.get() renvoie None pour le sommet de départ
    while courant is not None:
        chemin.append(courant)
        courant = parent.get(courant) # None si pas de parent (retour au départ)

    chemin.reverse() # On a remonté à l'envers : on renverse
    return chemin
```

Utilisation :

```
print(bfs_chemin('A', 'D', graphe)) # Renvoie ['A', 'B', 'D'] ou ['A', 'C', 'D']
```

Remarque : risque-t-on d'écraser un parent par un autre ? ▼

Non. `parent[voisin]` n'est assigné que si `not vu[voisin]` est vrai, c'est-à-dire si le voisin n'a pas encore été visité. Dans la foulée, `vu[voisin]` est alors immédiatement mis à `True`. Donc un sommet est enfilé — et son parent enregistré — *une seule fois*, la première fois qu'on le découvre.

Remarque : efficacité de `fil.pop(0)` ▼

`list.pop(0)` est en $\mathcal{O}(n)$ car Python décale tous les éléments restants. Pour un usage intensif, on préférerait `collections.deque` avec `popleft()`, qui travaille en $\mathcal{O}(1)$. En NSI, `list.pop(0)` est accepté, mais il est utile de savoir que ce n'est pas optimal.

Remarque : au-delà du plus court chemin ▼

Le BFS garantit le plus court chemin **en nombre d'arêtes** — ce qui suppose que toutes les liaisons ont le même coût.

Pour des graphes **pondérés** (routes avec distances réelles, réseaux avec débits...), il faut des algorithmes plus avancés, comme **Dijkstra** ou **Bellman-Ford**, qui ne sont pas au programme de NSI mais existent et sont largement utilisés (protocoles RIP ou OSPF, pour le routage sur Internet, GPS...).

4.4. Détection de cycle

Un graphe contient un cycle si, lors d'un parcours DFS (en profondeur), on retrouve un sommet qu'on est **en train d'explorer**. La difficulté est dans le « en train de visiter ». Avec un marquage binaire (visité / non visité), on ne sait pas *pourquoi* un sommet est déjà marqué. Il y a pourtant deux situations très différentes :

- on retombe sur un sommet dont **le DFS n'est pas encore revenu** (les appels récursifs n'ont pas encore été tous « épuisés » — on est encore « dedans ») : c'est un **vrai cycle** ;
- on arrive sur un sommet dont **le DFS est déjà revenu complètement** (tous les appels récursifs possibles ont été achevés). Ce sommet a donc été atteint par un autre chemin : **on n'a pas de cycle ici**.

Le marquage binaire confond les deux. Il faut donc utiliser **trois états** :

État	Valeur	Signification
Non visité	0	Le DFS n'est pas encore passé par ce sommet
En cours	1	Le DFS est entré dans ce sommet mais n'en est pas encore revenu (des appels récursifs sont encore en attente au-dessus de lui dans la pile d'appels)
Terminé	2	Le DFS est entré dans ce sommet et en est complètement revenu — tous ses voisins ont été explorés

Note : un sommet passe à l'état **2 uniquement au moment où l'appel dfs(sommet) est sur le point de se terminer** — c'est-à-dire après que tous les appels récursifs sur ses voisins ont eux-mêmes abouti.

Exemple : différence entre marquage binaire et ternaire ▼

Reprenons l'exemple de graphe ci-dessus. Rappelons que ce graphe comporte 4 sommets et deux chemins, **A→B→D** et **A→C→D** : il ne comporte pas de cycle.

Voici ce qu'il se passerait avec un marquage binaire (à gauche), et un marquage ternaire (à droite).

Marquage binaire	Marquage ternaire
<pre>dfs(A) : vu[A]=True dfs(B) : vu[B]=True dfs(D) : vu[D]=True ← retour ← retour dfs(C) : vu[C]=True voisin D : vu[D] == True → cycle détecté, mais ❌</pre> FAUX positif	A→B→D, A→C→D (pas de cycle) <pre>dfs(A) : etat[A]=1 dfs(B) : etat[B]=1 dfs(D) : etat[D]=1 ← retour : etat[D]=2 ← retour : etat[B]=2 dfs(C) : etat[C]=1 voisin D : etat[D] == 2 → ignoré ✓ (déjà terminé) ← retour : etat[C]=2 ← retour : etat[A]=2 → aucun cycle détecté ✅</pre> Aucun cycle détecté

Voici un code Python, qui travaille sur un graphe codé sous forme de liste / dictionnaire d'adjacence. Il exploite une **fonction imbriquée** (*nested function* en anglais) : elle n'est accessible *que* depuis la fonction qui la contient. Elle a accès en *lecture* aux variables de la fonction englobante sans qu'elles lui soient passées en paramètre. Elle peut également *modifier* le contenu d'une variable englobante si celle-ci est *mutable* (comme un dictionnaire ou une liste). C'est précisément ce qu'on fait ici avec `etat`.

```
def contient_cycle(graphe):
    # 0 = non visité, 1 = en cours (DFS pas encore revenu), 2 = terminé
    etat = {sommet: 0 for sommet in graphe}

    def dfs(sommet):
        etat[sommet] = 1
        for voisin in graphe[sommet]:
            if etat[voisin] == 1:          # DFS n'est pas encore revenu de ce voisin
                return True              # → on a donc trouvé un cycle !
            # Appel récursif uniquement sur les voisins non encore visités
            if etat[voisin] == 0 and dfs(voisin):
                # dfs(voisin) a renvoyé True : cycle détecté plus profond dans la récursion
                return True              # → on "remonte" donc l'information !
        etat[sommet] = 2                  # Tous les voisins traités : le sommet a été "épuisé"
        return False

    for sommet in graphe:
        if etat[sommet] == 0:
            if dfs(sommet):
                return True
    return False
```

Retour sur la note : voyez où, dans le code, un sommet passe à l'état 2. C'est à la *dernière instruction* de son propre appel `dfs`, **après** que tous ses voisins ont été traités, mais **avant** que la fonction ne rende la main à son appelant.

⚠ Graphe orienté vs non orienté

Dans un graphe **non orienté**, le marquage ternaire n'est pas nécessaire. Le risque est différent : depuis un sommet `u`, on « voit » toujours son parent immédiat (puisque l'arête est bidirectionnelle) — il ne faut pas le confondre avec un cycle. Il suffit de passer le parent en paramètre et de l'exclure des voisins à vérifier.

5. Points d'attention

⚠ Erreurs classiques

- Oublier le marquage des sommets visités → boucle infinie sur tout graphe avec cycle.
- Définir `vu` à l'intérieur d'un DFS récursif → le dictionnaire est réinitialisé à chaque appel, le marquage est perdu.
- Confondre DFS et BFS : le DFS ne garantit pas le chemin le plus court ; le BFS le garantit (uniquement en l'absence de poids sur les arêtes).
- Confondre arête et arc : arête pour un graphe non orienté, arc pour un graphe orienté.
- Se tromper dans l'interprétation de `matrice[i][j]` : `i` est la *source*, `j` la *destination* — pas l'inverse.

6. Liens avec d'autres notions

- **Arbres** : un arbre est un graphe non orienté, connexe et sans cycle. Les algorithmes sur les arbres (parcours en profondeur, en largeur) sont des cas particuliers des algorithmes sur les graphes — sans avoir à gérer les cycles.

- **Piles et files** : structures auxiliaires indispensables aux parcours DFS (pile) et BFS (file).
- **Récursivité** : le DFS récursif utilise implicitement la pile d'appels comme pile auxiliaire.
- **Complexité** : les parcours DFS et BFS ont une complexité en $\mathcal{O}(S + A)$ avec une liste d'adjacence — linéaire en la taille du graphe.
- **Modélisation** : réseaux informatiques, réseaux sociaux, cartes routières, dépendances entre tâches (graphe orienté sans cycle, dit DAG).

✓ Récapitulatif — ce qu'il faut maîtriser

- Distinguer graphe orienté / non orienté, arête / arc.
- Lire et construire une liste d'adjacence (dictionnaire Python) et une matrice d'adjacence.
- Comprendre pourquoi le marquage des sommets visités est indispensable.
- Écrire ou compléter un DFS récursif avec dictionnaire vu externe.
- Écrire ou compléter un BFS itératif avec file et reconstruction de chemin.
- Savoir que BFS garantit le plus court chemin (en arêtes), pas DFS.